

# Data Preparation for Exploration

Session 4

## Agenda

- Recap
- Joins
  - Inner Join
  - Outer Join
  - Right Join
  - Left Join
  - Self Join
  - Cartesian Product
- Practice On Joins
- Aggregation
- Grouping

# Types of Join

- **Equi JOIN:** An **Equi JOIN** is a type of SQL join that combines rows from two or more tables based on a **condition of equality** between a **primary key (PK)** in one table and a **foreign key (FK)** in another table.
- **Non-Equi JOIN:** A **Non-Equi JOIN** is a type of SQL join where the condition used to join the tables is not based on equality between a primary key and a foreign key. Instead, it involves a different relationship, such as a range condition or an inequality.

# Equijoins and Non-Equijoins

- Equi join: when the join condition is based on the equality operator

```
SELECT e.emp_id, e.last_name, d.dept_name  
FROM employees e  
JOIN departments d ON e.deptno = d.dno;
```

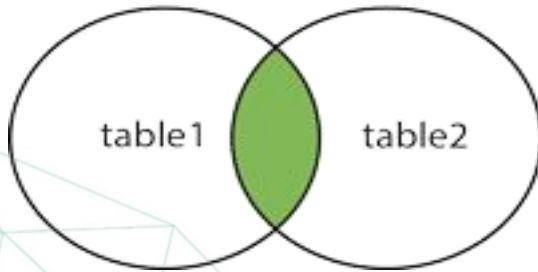
- Non-Equijoin: when the join condition is based on any operator rather than the equality operator

```
SELECT e.emp_id, e.last_name, s.range_name  
FROM employees e  
JOIN salary_ranges s ON e.salary BETWEEN s.min_salary AND s.max_salary;
```

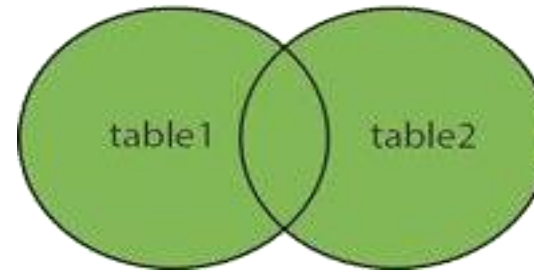
# Joins

A JOIN is used to combine rows from two or more tables, based on a related column between them.

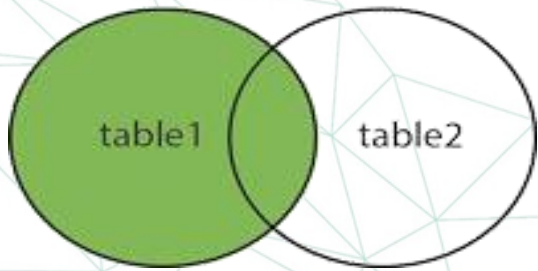
INNER JOIN



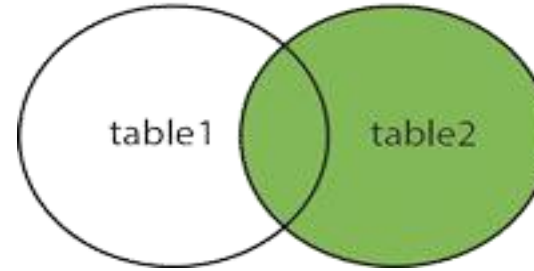
FULL OUTER JOIN



LEFT JOIN



RIGHT JOIN



# Types of Equi Join

- **INNER JOIN:** Returns rows when there is at least one match in both tables
- **Outer JOIN:** Returns matched rows as well as rows when there is no match in one of the tables
- **Self JOIN:** Write the same table many times in the FROM and join itself
- **Cartesian Product (Cross Join):** Returns all possible combinations of rows

# INNER JOIN

**INNER JOIN** returns only the rows where there is a match in both tables based on the specified join condition. It filters out rows that do not have corresponding matches in the joined tables.

## Syntax

```
SELECT table1.column1, table2.column2 FROM table1  
INNER JOIN table2  
ON table1.column = table2.column
```

inner\_join(x, y)

|   |    |   |    |
|---|----|---|----|
| 1 | x1 | 1 | y1 |
| 2 | x2 | 2 | y2 |
| 3 | x3 | 4 | y4 |

inner\_join(x, y)

|   |    |    |
|---|----|----|
| 1 | x1 | y1 |
| 2 | x2 | y2 |

# Join Syntax (Inner Join)

```
SELECT table1.column1, table2.column2  
FROM table1  
INNER JOIN table2  
ON table1.column = table2.column
```

```
SELECT employees.emp_id, employees.last_name, departments.dept_name  
FROM employees  
INNER JOIN departments ON employees.deptno = departments.dno;
```

# Ambiguous Column Names

- When querying two or more tables that have columns with the same name, you encounter ambiguous column names. This happens when both tables have columns with identical names, such as "name" or "ID", making it unclear which table's column the query is referring to.
- **Solution:**
  - To resolve this ambiguity, you should **prefix the ambiguous column names with the table name** or its alias. This practice not only clarifies the source of each column but also ensures that the query runs correctly.
- **Best Practice:**
  - It is generally recommended to **prefix all column names with the table name or alias** even when there's no ambiguity. This approach improves code readability and reduces potential errors, especially in complex queries involving multiple tables.

# Ambiguous Column Names

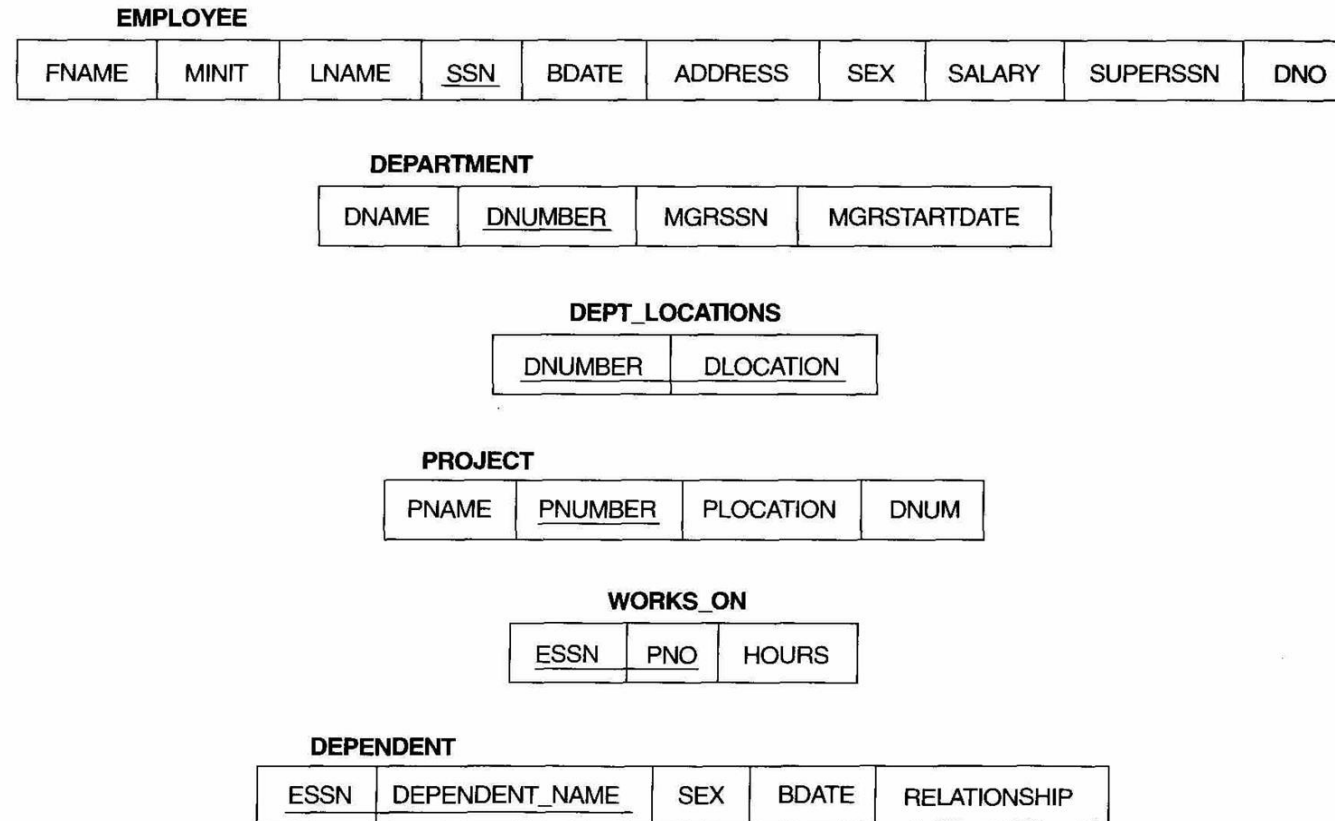
```
SELECT employees.ID, employees.name, departments.name  
FROM employees  
INNER JOIN departments ON employees.deptno = departments.dno;
```

## Explanation:

- **employees.ID** refers to the ID column from the employees table.
- **employees.name** refers to the name column from the employees table.
- **departments.name** refers to the name column from the departments table.

# Example

- Query All employees working for any department.



**Figure 7.5** Schema diagram for the COMPANY relational database schema; the primary keys are underlined.

# Example

- Query All employees working for any department.

```
SELECT department.name,  
       employee.ID,  
       employee.name,  
       employee.Salary  
FROM   department  
JOIN   employee  
ON     department.id = employee.deptid  
ORDER BY department.name;
```

# Table Alias

- **Definition:** A **Table Alias** is an alternative name given to a table in a SQL query. This alias is often used to shorten long or complex table names, making the query easier to write and read. It can also be used to resolve ambiguity when multiple tables in a query have columns with the same name.
- **Benefits of Using Table Aliases:**
  - **Simplicity:** Aliases simplify queries, especially when dealing with long table names.
  - **Clarity:** They make the query more readable and concise.
  - **Ambiguity Resolution:** When columns from different tables have the same name, aliases help to distinguish between them.

# Outer Join

- **LEFT JOIN:** Return all rows from the left table, even if there are no matches in the right table
- **RIGHT JOIN:** Return all rows from the right table, even if there are no matches in the left table
- **FULL JOIN:** Return rows when there is a match in one of the tables

returns all records from the left table (table1), and the matching records from the right table (table2)

## Syntax

```
SELECT table1.column1, table2.column2 FROM table1  
LEFT JOIN table2  
ON table1.column = table2.column
```

left\_join(x, y)

|   |    |   |    |
|---|----|---|----|
| 1 | x1 | 1 | y1 |
| 2 | x2 | 2 | y2 |
| 3 | x3 | 4 | y4 |

left\_join(x, y)

|   |    |    |
|---|----|----|
| 1 | x1 | y1 |
| 2 | x2 | y2 |
| 3 | x3 |    |

returns all records from the right table (table2), and the matching records from the left table (table1)

## Syntax

```
SELECT table1.column1, table2.column2 FROM table1  
RIGHT JOIN table2  
ON table1.column = table2.column
```

returns all records when there is a match in left (table1) or right (table2) table records

## Syntax

```
SELECT table1.column1, table2.column2 FROM table1  
OUTER JOIN table2  
ON table1.column = table2.column WHERE condition
```

# Outer Join

- Display all departments' information even if they have no employees assigned.

```
SELECT e.name AS Employee, d.dept_id,  
       d.name AS Department  
FROM   employees e  
RIGHT OUTER JOIN departments d  
ON     e.dept_id = d.id;
```

| Employees     | Dept_ID | Departments |
|---------------|---------|-------------|
| Ahmed Ali     | 100     | IT          |
| Mohamed Samir | 200     | Marketing   |
| Mona Selim    | 100     | IT          |
|               | 300     | HR          |

# SELF JOIN

- A **SELF JOIN** is a regular join, but the table is joined with itself. This can be useful when you need to compare rows within the same table.

```
SELECT t1.column AS column1, t2.column AS column2  
FROM table AS t1, table AS t2  
ON t1.column = t2.column;
```

- **SELECT t1.column AS column1, t2.column AS column2:** Selects columns from two different instances of the same table.
- **FROM table AS t1, table AS t2:** The table is referenced twice with different aliases (t1 and t2).
- **ON t1.column = t2.column:** Defines the condition for the join, comparing columns within the same table.

- Query the employees table to list name of each employee and his supervisor's name.

```
SELECT e.name Employee_name , s.name Supervisor  
FROM employees e , employees s  
WHERE e.supervisorID = s.ID
```

# Self Join

- Scenario:** An organization is organizing a Christmas party and selecting Secret Santas among employees using a color-picking method.
- Process:**
  - Each employee is assigned a unique color.
  - Employees pick a color from a pool, which determines their Secret Santa.

```
SELECT e1.EmployeeID AS Santa, e2.EmployeeID AS Recipient
FROM Employees e1
JOIN Employees e2 ON e1.PickedColor = e2.AssignedColor;
```

| Color | Name  | Color |
|-------|-------|-------|
| Blue  | John  | Red   |
| Green | Alex  | Blue  |
| Red   | Simon | Green |



| Color | Name  | Color |
|-------|-------|-------|
| Blue  | John  | Red   |
| Green | Alex  | Blue  |
| Red   | Simon | Green |

| Name  | Secret_Santa |
|-------|--------------|
| John  | Simon        |
| Alex  | John         |
| Simon | Alex         |

# Self Join

| ID   | Employee_name | SupervisorID |
|------|---------------|--------------|
| 1122 | Ahmed Ali     | 2233         |
| 2233 | Kamel Mohamed | 1234         |
| 1234 | Hanaa Sobhy   | 2233         |
| 3216 | Amr Omran     | NULL         |
| 9685 | Noha Mohamed  | 2233         |
| 5124 | Edward Hanna  | 2233         |
| 6699 | Mariam Adel   | 5124         |
| 5216 | Maged Raoof   | 9685         |

# Example (Non-Equijoin)

- Display employee's information ( name , salary , title) with salary grade for each employee.

```
SELECT e.name, e.salary, j.grade
FROM employees e
JOIN job_grades j
ON e.salary BETWEEN j.lowsal AND j.highsal;
```

job\_grades

| LowSal | HighSal | Grade |
|--------|---------|-------|
| 1000   | 4000    | B     |
| 4000   | 7000    | A     |

employees

| EmpNo | Name  | Salary |
|-------|-------|--------|
| 100   | Ahmed | 5000   |
| 200   | Mai   | 3000   |

# Let's Practice

# Exercise #1

Table: Employees

| Column Name | Type    |
|-------------|---------|
| id          | int     |
| name        | varchar |

id is the primary key (column with unique values) for this table.  
Each row of this table contains the id and the name of an employee in a company.

Table: EmployeeUNI

| Column Name | Type |
|-------------|------|
| id          | int  |
| unique_id   | int  |

(id, unique\_id) is the primary key (combination of columns with unique values) for this table.  
Each row of this table contains the id and the corresponding unique id of an employee in the company.

Write a solution to show the **unique ID** of each user, If a user does not have a unique ID replace just show `null`.

Return the result table in **any** order.

The result format is in the following example.

# Example

## Input:

Employees table:

| id | name     |
|----|----------|
| 1  | Alice    |
| 7  | Bob      |
| 11 | Meir     |
| 90 | Winston  |
| 3  | Jonathan |

EmployeeUNI table:

| id | unique_id |
|----|-----------|
| 3  | 1         |
| 11 | 2         |
| 90 | 3         |

## Output:

| unique_id | name     |
|-----------|----------|
| null      | Alice    |
| null      | Bob      |
| 2         | Meir     |
| 3         | Winston  |
| 1         | Jonathan |

## Exercise #2

Table: Sales

| Column Name | Type |
|-------------|------|
| sale_id     | int  |
| product_id  | int  |
| year        | int  |
| quantity    | int  |
| price       | int  |

(sale\_id, year) is the primary key (combination of columns with unique values) of this table.  
product\_id is a foreign key (reference column) to Product table.  
Each row of this table shows a sale on the product product\_id in a certain year.  
Note that the price is per unit.

Table: Product

| Column Name  | Type    |
|--------------|---------|
| product_id   | int     |
| product_name | varchar |

product\_id is the primary key (column with unique values) of this table.  
Each row of this table indicates the product name of each product.

Write a solution to report the product\_name, year, and price for each sale\_id in the Sales table.

# Example

## Input:

Sales table:

| sale_id | product_id | year | quantity | price |
|---------|------------|------|----------|-------|
| 1       | 100        | 2008 | 10       | 5000  |
| 2       | 100        | 2009 | 12       | 5000  |
| 7       | 200        | 2011 | 15       | 9000  |

Product table:

| product_id | product_name |
|------------|--------------|
| 100        | Nokia        |
| 200        | Apple        |
| 300        | Samsung      |

## Output:

| product_name | year | price |
|--------------|------|-------|
| Nokia        | 2008 | 5000  |
| Nokia        | 2009 | 5000  |
| Apple        | 2011 | 9000  |

## Explanation:

From sale\_id = 1, we can conclude that Nokia was sold for 5000 in the year 2008.

From sale\_id = 2, we can conclude that Nokia was sold for 5000 in the year 2009.

From sale\_id = 7, we can conclude that Apple was sold for 9000 in the year 2011.

## Exercise #3

Table: Visits

| Column Name | Type |
|-------------|------|
| visit_id    | int  |
| customer_id | int  |

visit\_id is the column with unique values for this table.

This table contains information about the customers who visited the mall.

Table: Transactions

| Column Name    | Type |
|----------------|------|
| transaction_id | int  |
| visit_id       | int  |
| amount         | int  |

transaction\_id is column with unique values for this table.

This table contains information about the transactions made during the visit\_id.

Write a solution to find the IDs of the users who visited without making any transactions and the number of times they made these types of visits.

Return the result table sorted in **any order**.

# Example

## Input:

### Visits

| visit_id | customer_id |
|----------|-------------|
| 1        | 23          |
| 2        | 9           |
| 4        | 30          |
| 5        | 54          |
| 6        | 96          |
| 7        | 54          |
| 8        | 54          |

### Transactions

| transaction_id | visit_id | amount |
|----------------|----------|--------|
| 2              | 5        | 310    |
| 3              | 5        | 300    |
| 9              | 5        | 200    |
| 12             | 1        | 910    |
| 13             | 2        | 970    |

## Output:

| customer_id | count_no_trans |
|-------------|----------------|
| 54          | 2              |
| 30          | 1              |
| 96          | 1              |

# Aggregate Functions

- Aggregate Functions in SQL are used to perform calculations on a set of values and return a single value. They are commonly used with the GROUP BY clause to aggregate data based on specified columns.

# Aggregate Functions

- Aggregate Functions (group functions): perform a specific operation on many rows and return one result per group
- Group Functions ignore Null values in the columns except in **COUNT(\*)**

# Aggregate Functions

| Function | Description                               |
|----------|-------------------------------------------|
| AVG      | return the average value.                 |
| COUNT    | return the number of values.              |
| MAX      | return the maximum value.                 |
| MIN      | return the minimum value.                 |
| SUM      | return the sum of all or distinct values. |

# Examples

Find the sum, maximum, minimum and average salaries of all employees

```
SELECT  
    SUM(salary) AS TotalSalary,  
    MAX(salary) AS HighestSalary,  
    MIN(salary) AS LowestSalary,  
    AVG(salary) AS AverageSalary  
FROM Employees;
```

Find the total number of employees in the company?

```
SELECT COUNT(*) AS TotalEmployees  
FROM employees;
```

Find the number of employees in the research department?

```
SELECT COUNT(*) AS NumberOfEmployeesInResearch  
FROM employees  
WHERE department = 'Research';
```

# Grouping

- If you want to apply aggregate functions to subgroups of tuples, use **GROUP BY** clause
- **GROUP BY** clause must be used in conjunction with aggregate functions
- You can **filter** group results using **HAVING** clause
- **HAVING** clause is used for applying conditions on group functions

# GROUP BY

- **GROUP BY** clause is used in a **SELECT** statement to group the results by one or more columns.

```
SELECT column_1, column_2,  
       aggregate_function(column_3)  
FROM table_name  
GROUP BY column_1, column_2,
```

# Grouping Examples

- For each department, retrieve the department number, the number of employees in the department, and their average salaries

```
SELECT dno AS DepartmentNumber,  
       COUNT(*) AS NumberOfEmployees,  
       AVG(salary) AS AverageSalary  
FROM employee  
GROUP BY dno;
```

# Using DISTINCT with Aggregate Functions

- Use DISTINCT with aggregate functions to **summarize** only unique values
- DISTINCT aggregates **eliminate** duplicate values, not rows (unlike SELECT DISTINCT)
- **Compare** (with partial results)

# Using DISTINCT with Aggregate Functions

```
SELECT empid,  
       YEAR(orderdate) AS orderyear,  
       COUNT(custid) AS all_custs,  
       COUNT(DISTINCT custid) AS unique_custs  
FROM Sales.Orders  
GROUP BY empid, YEAR(orderdate);
```

| empid | orderyear | all_custs | unique_custs |
|-------|-----------|-----------|--------------|
| 101   | 2023      | 150       | 120          |
| 101   | 2024      | 200       | 180          |
| 102   | 2023      | 180       | 150          |
| 102   | 2024      | 220       | 190          |

# Using Aggregate Functions with NULL

- Most aggregate functions ignore NULL
  - COUNT(<column>) ignores NULL
  - COUNT(\*) counts all rows
- NULL may produce incorrect results (such as use of AVG)
- Use ISNULL or COALESCE to replace NULLs before aggregating

```
SELECT  
    AVG(c2) AS AvgWithNULLs,  
    AVG(COALESCE(c2,0)) AS AvgWithNULLReplace  
FROM dbo.t2;
```

# HAVING

- HAVING clause is used in combination with the GROUP BY clause to restrict the groups of returned rows to only those whose condition is TRUE

```
SELECT column1, column2, agg_func(column3)
FROM table_name
WHERE condition
GROUP BY column1, column2
HAVING condition
```

# Grouping with Having Examples

- For each project on which more than two employees work, retrieve the project number, the project name , and the number of employees who work on the project

```
SELECT pnumber, pname, COUNT(works_on.pno) AS num_employees
FROM project
JOIN works_on ON project.pnumber = works_on.pno
GROUP BY pnumber, pname
HAVING COUNT(*) > 2
ORDER BY pnumber;
```

# SQL Execution Order

1. **FROM and JOINS**
2. **WHERE**
3. **GROUP BY**
4. **Aggregate Functions**
5. **HAVING**
6. **SELECT**
7. **ORDER BY**

# GROUP BY and the Logical Order of Operations

- If a query uses GROUP BY, all subsequent phases operate on the groups, not source rows
- HAVING, SELECT, and ORDER BY must return a single value per group
- All columns in SELECT, HAVING, and ORDER BY must appear in GROUP BY clause or be inputs to aggregate expressions

# GROUP BY Workflow

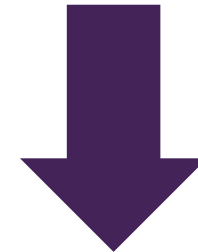
```
SELECT orderid, empid, custid
FROM Sales.Orders;
```

| orderid | empid | custid |
|---------|-------|--------|
| 10643   | 6     | 1      |
| 10692   | 4     | 1      |
| 10926   | 4     | 2      |
| 10625   | 3     | 2      |
| 10365   | 3     | 3      |



```
WHERE
custid IN(1,2)
```

| orderid | empid | custid |
|---------|-------|--------|
| 10643   | 6     | 1      |
| 10692   | 4     | 1      |
| 10926   | 4     | 2      |
| 10625   | 3     | 2      |



```
GROUP BY empid
```

| empid | COUNT(*) |
|-------|----------|
| 6     | 1        |
| 4     | 2        |
| 3     | 1        |

# Any Questions ?

# Thank You